Revista Eletrônica de Iniciação Científica em Computação
ISSN: 3085-8461 • doi: 10.5753/reic.202X.XXXXXX
This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Artigo de Pesquisa/Research Paper

Análise do Impacto de um Pré-Processamento Simétrico $O(n)$ na Redução de Inversões e Eficiência de Algoritmos de Ordenação

Analyzing the Impact of an $O(n)$ Symmetric Preprocessing on Inversion Reduction and Sorting Algorithm Efficiency

Samuel da Penha Nascimento [Universidade Estadual do Piauí | *samuelpnascimentodev@gmail.com*]
Francisco das Chagas Rocha [Universidade Estadual do Piauí |*rocha@phb.uespi.br*]

Coordenação do Curso de Sistemas de Computação, Universidade Estadual do Piauí, Campus Prof. Alexandre Alves de Oliveira, Parnaíba, Piauí – Brasil.

Resumo. Algoritmos quadráticos são úteis em sistemas com recursos restritos, porém sensíveis às inversões na entrada. Todavia, a eficácia de reduzir essas inversões via pré-processamento em diferentes classes de algoritmos é pouco explorada. Este artigo avalia o impacto do pré-processamento de inversões em algoritmos adaptativos e não adaptativos. Para tanto, compararam-se oito métodos em dez topologias de entrada com vetores de até 10^7 elementos. O pré-processamento reduziu até 62% das inversões em entradas desalinhadas e 100% no Invertido (99,99% no Zigue-zague) em $n{=}10^4$, gerando speedups de até $\times 2.280$ em métodos adaptativos (em $n{=}10^4$), mas degradando o Quicksort na implementação avaliada (três vias com mediana de três) e mantendo-se inferior aos algoritmos padrão em entradas aleatórias (métodos quadráticos até $n{=}10^5$), ao passo que entradas *alinhadas* (Invertido, Zigue-zague) apresentam geometria de desordem compatível com a varredura simétrica e as demais são *desalinhadas*.

Abstract. Quadratic algorithms are useful in resource-constrained systems, yet sensitive to input inversions. However, the efficacy of reducing these inversions via preprocessing across different algorithm classes remains poorly explored. This paper evaluates the impact of inversion preprocessing on adaptive and non-adaptive algorithms. To this end, eight methods were compared across ten input topologies with vectors of up to 10^7 elements. Preprocessing reduced up to 62% of inversions in unaligned inputs and 100% in Inverted (99.99% in Zigzag) at $n{=}10^4$, yielding speedups of up to $\times 2{,}280$ in adaptive methods (at $n{=}10^4$), while degrading Quicksort in the evaluated implementation (three-way partitioning with median-of-three) and remaining inferior to standard algorithms on random inputs (quadratic methods up to $n{=}10^5$), whereas *aligned* inputs (Inverted, Zigzag) have disorder geometry matching the symmetric scan and the rest are *unaligned*.

Palavras-chave: Ordenação, Pré-processamento simétrico, Inversões, Algoritmos adaptativos

Keywords: Sorting, Symmetric preprocessing, Inversions, Adaptive algorithms

Recebido/Received: DD Month YYYY • **Aceito/Accepted:** DD Month YYYY • **Publicado/Published:** DD Month YYYY

1 Introdução

A ordenação de dados é uma operação fundamental na Ciência da Computação, sendo essencial em sistemas de busca, bancos de dados e aplicações em tempo real [Knuth, 1973; Sedgewick and Wayne, 2011]. Embora métodos com complexidade assintótica $O(n \log n)$, como Merge Sort e Quicksort, sejam preferíveis para o tratamento de grandes volumes de dados, algoritmos de natureza quadrática $O(n^2)$, como Insertion Sort, Selection Sort e Bubble Sort, permanecem relevantes em cenários específicos [Cormen *et al*., 2022]. Essa permanência está relacionada à simplicidade estrutural, ao baixo consumo de memória por operarem *in-place* e à eficiência prática em conjuntos de dados pequenos ou parcialmente ordenados [Cormen *et al*., 2022; Ziviani, 2021].

A eficiência prática de um algoritmo de ordenação, entretanto, não depende exclusivamente de sua complexidade assintótica, sendo também influenciada pelas propriedades da sequência de entrada [Estivill-Castro and Wood, 1992]. Nesse contexto, a inversão, definida como um par de elementos que satisfaz $A[i] > A[j]$ para $i < j$, constitui uma medida quantitativa do grau de desordem de um arranjo [Knuth, 1973; Mannila, 1985]. O número de inversões permite caracterizar diferentes níveis de ordenação prévia (*presortedness*), uma vez que sequências com maior quantidade de inversões apresentam maior desordem em relação à ordem final [Mannila, 1985]. Essa característica pode influenciar significativamente o custo de execução de determinados algoritmos, especialmente daqueles cuja quantidade de operações depende do grau de desordem da entrada [Estivill-Castro and Wood, 1992].

Apesar dessa relação entre desordem da entrada e custo de execução, a sensibilidade às condições iniciais varia entre os métodos quadráticos: o *Insertion Sort* é adaptativo ao número de inversões I [Mannila, 1985; Estivill-Castro and Wood, 1992], o *Bubble Sort* com saída antecipada é adaptativo ao deslocamento máximo (não estritamente a I), e o *Selection Sort* é não adaptativo. Em particular, entradas com elevada quantidade de inversões aumentam o número de movimentações do *Insertion Sort* [Cormen *et al*., 2022]. Nesse cenário, uma etapa preliminar capaz de reduzir a desordem da entrada pode diminuir o trabalho posterior da ordenação.

A relevância deste estudo está na investigação de uma

estratégia capaz de melhorar o comportamento prático de algoritmos de ordenação sem modificar sua estrutura fundamental ou introduzir elevado custo computacional adicional. A redução do grau de desordem antes da ordenação principal pode diminuir o número de operações realizadas em entradas desfavoráveis, especialmente em métodos sensíveis à quantidade de inversões [Estivill-Castro and Wood, 1992; Hwang *et al*., 2000]. Além disso, o estudo contribui para a análise de algoritmos adaptativos ao investigar as condições nas quais o custo do pré-processamento é compensado pela redução do trabalho realizado durante a ordenação [Mannila, 1985; Hwang *et al*., 2000].

Formalmente, denota-se por C_{original} o custo computacional temporal da ordenação direta da entrada. Ao introduzir o pré-processamento simétrico com custo C_{pre}, o algoritmo de ordenação subsequente passa a atuar sobre um vetor modificado, despendendo um tempo reduzido C_{sort}. A lacuna abordada é a eficácia de reduzir inversões via pré-processamento $O(n)$ entre diferentes classes de algoritmos, pouco explorada experimentalmente. O objetivo deste trabalho é propor, implementar e avaliar um algoritmo simétrico de pré-processamento de baixo custo $O(n)$ para reduzir inversões. Busca-se verificar experimentalmente se a abordagem híbrida é temporalmente mais eficiente que a aplicação direta dos algoritmos convencionais, identificando empiricamente as condições práticas em que $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$ se sustenta. As contribuições são: (i) o algoritmo de varredura simétrica; (ii) um benchmark com oito métodos em dez topologias; e (iii) uma regra prática de viabilidade para o *Insertion Sort*. A comparação inclui os ordenadores da biblioteca padrão do Rust (`sort_unstable/sort`) e uma detecção $O(n)$ de *run* descendente com reversão; uma teoria fechada do valor esperado de redução permanece como trabalho futuro.

Para avaliar a eficácia do pré-processamento proposto, realizou-se um estudo experimental com uma implementação em Rust. A abordagem foi submetida a benchmarks automatizados contra algoritmos de ordenação consolidados na literatura e os ordenadores da biblioteca padrão. A avaliação abrangeu vetores sintéticos sob variados níveis de inversão — do pior caso a topologias adversariais — bem como um prefixo de bytes de um asset congelado (rótulo operacional "Real", sem pretensão de representatividade geral; Seção 3.1). Não se medem consumo de memória ou energia: o foco é o tempo de execução.

Este artigo está estruturado em seis seções, além desta introdução. A Seção 2 apresenta o Referencial Teórico. A Seção 3 descreve os Métodos e Técnicas da Pesquisa, a formalização do algoritmo de pré-processamento proposto, dos algoritmos de ordenação avaliados e os procedimentos experimentais. A Seção 4 apresenta e discute os resultados obtidos. A Seção 5 apresenta as Considerações Finais. Por fim, a Seção 6 declara o uso de Inteligência Artificial e a Seção 7 disponibiliza o repositório da pesquisa.

2 Referencial Teórico

Nesta seção, são apresentados os fundamentos teóricos necessários à compreensão da pesquisa, abrangendo o conceito de inversões como medida de desordem, os princípios da ordenação adaptativa e as características dos algoritmos avaliados, contemplando métodos quadráticos e quasilineares. Em seguida, são discutidos o conceito de pré-processamento, os princípios do Pré-processamento Simétrico e sua finalidade na redução de inversões, além dos trabalhos relacionados que fundamentam teoricamente a abordagem adotada e situam a contribuição desta pesquisa no contexto da literatura. Esses fundamentos estabelecem as bases para compreender a relação entre a estrutura da entrada, o comportamento dos algoritmos de ordenação e os efeitos esperados da aplicação do pré-processamento proposto.

2.1 Inversões, Ordenação Adaptativa e Algoritmos Avaliados

A eficiência prática dos algoritmos de ordenação tradicionais frequentemente depende da disposição inicial dos elementos no conjunto de dados de entrada [Cormen *et al*., 2022; Estivill-Castro and Wood, 1992]. Para formalizar e quantificar o grau de desordem de uma sequência, utiliza-se classicamente o conceito de *inversão* [Knuth, 1973]. Dado um vetor a composto por n elementos, uma inversão é definida formalmente como um par de índices (i, j) tal que $i < j$ e $a[i] > a[j]$ [Knuth, 1973; Sedgewick and Wayne, 2011]. O número total de inversões I constitui uma medida do grau de desordem da entrada e está diretamente relacionado ao esforço necessário para ordená-la por meio de operações locais [Mannila, 1985].

A relação entre o volume de inversões I e a complexidade de execução dá origem à propriedade de *adaptabilidade* em algoritmos de ordenação [Estivill-Castro and Wood, 1992]. Um algoritmo é considerado adaptativo quando sua complexidade temporal melhora sensivelmente na presença de um ordenamento prévio parcial na entrada [Mannila, 1985]. Um exemplo clássico é o *Insertion Sort*, cuja execução consome $O(n + I)$ operações. Em um vetor estritamente ordenado ($I = 0$), o algoritmo atinge tempo linear $O(n)$, enquanto, no pior caso ($I = O(n^2)$), degrada para tempo quadrático [Cormen *et al*., 2022; Knuth, 1973].

Para avaliar o impacto do pré-processamento simétrico sobre diferentes paradigmas de ordenação, selecionou-se um conjunto representativo de algoritmos com características distintas quanto à complexidade temporal, adaptabilidade e sensibilidade à ordem inicial dos elementos. A seleção foi realizada de modo a contemplar tanto algoritmos cuja eficiência pode ser diretamente influenciada pela quantidade de desordem presente na entrada quanto métodos cujo comportamento é menos dependente dessa propriedade.

No grupo de algoritmos com complexidade temporal quadrática $O(n^2)$ no pior caso, foram avaliados o *Bubble Sort*, o *Insertion Sort* e o *Selection Sort* [Cormen *et al*., 2022]. O *Insertion Sort* é adaptativo ao número de inversões e o *Bubble Sort* avaliado possui saída antecipada (adaptativo ao deslocamento máximo, não estritamente a I) [Estivill-Castro and Wood, 1992], o que os torna relevantes para analisar os impactos da redução de inversões. A motivação de sistemas com recursos restritos não é medida diretamente (sem métricas de memória ou energia). Em contrapartida, o *Selection Sort* é não adaptativo e mantém um número fixo de comparações independentemente da configuração inicial [Estivill-Castro and Wood, 1992], servindo como contraponto aos algoritmos sensíveis ao pré-processamento.

No grupo dos algoritmos de complexidade quasilinear,

foram avaliados o *Merge Sort*, cuja complexidade temporal é $O(n \log n)$, e o *Quicksort*, cujo desempenho médio é $O(n \log n)$, embora seu pior caso assintótico seja $O(n^2)$ [Cormen *et al*., 2022]. A inclusão desses métodos permite verificar se os efeitos observados nos algoritmos quadráticos permanecem relevantes quando o custo assintótico da ordenação é inferior. Dessa forma, o experimento não se restringe a algoritmos diretamente relacionados à contagem de inversões, permitindo analisar também a influência do pré-processamento em métodos cuja estratégia de ordenação apresenta características distintas.

Assim, o conjunto avaliado contempla três dimensões relevantes para a análise experimental: algoritmos quadráticos adaptativos (*Bubble Sort* e *Insertion Sort*), um algoritmo quadrático não adaptativo (*Selection Sort*) e algoritmos quasilineares baseados em estratégias distintas de ordenação (*Merge Sort* e *Quicksort*). Essa composição permite verificar não apenas se o pré-processamento simétrico reduz a quantidade de inversões da entrada, mas também em que medida essa redução se converte em ganhos de desempenho para algoritmos com diferentes graus de dependência em relação à ordem inicial dos dados.

2.2 Pré-Processamento Simétrico

O conceito de pré-processamento, no contexto do projeto e análise de algoritmos, refere-se a uma etapa preliminar de transformação ou reestruturação dos dados de entrada realizada antes da execução do algoritmo principal [Cormen *et al*., 2022]. O propósito dessa fase preparatória é alterar a disposição ou as propriedades da instância do problema, tornando a resolução subsequente mais simples ou computacionalmente mais eficiente. De forma conceitual e intuitiva, o pré-processamento atua de maneira análoga à organização prévia de um ambiente ou material de trabalho antes da execução de uma tarefa complexa: embora a etapa de preparação exija um custo computacional inicial, a redução no esforço exigido no processamento principal compensa esse investimento prévio.

Para que uma etapa de pré-processamento seja viável em problemas de ordenação, seu custo temporal deve ser estritamente dominado pela complexidade do algoritmo de ordenação que o sucede [Cormen *et al*., 2022; Sedgewick and Wayne, 2011]. Dessa forma, um pré-processamento ideal deve apresentar complexidade temporal linear $O(n)$, garantindo que o *overhead* introduzido seja assintoticamente desprezível em relação ao custo total da ordenação — seja em algoritmos quasilineares $O(n \log n)$ no caso médio ou quadráticos $O(n^2)$ no pior caso [Knuth, 1973].

Especificamente nesta pesquisa, a etapa de pré-processamento é denominada *simétrica* por aplicar uma varredura bidirecional e espelhada a partir das extremidades do vetor. Essa reestruturação visa reduzir o número total de inversões I da entrada ao mover elementos discrepantes para posições mais próximas de seus destinos finais em um único passe de complexidade $O(n)$. Como resultado, ao fornecer uma estrutura parcialmente reordenada para os algoritmos subsequentes, busca-se mitigar o número de comparações e trocas operadas tanto por métodos adaptativos quanto por métodos não adaptativos.

2.3 Trabalhos Relacionados

A literatura apresenta diferentes abordagens para caracterizar o grau de ordenação de uma sequência e explorar essa informação no processo de ordenação. O trabalho seminal de Mannila [1985] teve como objetivo definir conceitualmente a pré-ordenação e estabelecer medidas formais capazes de quantificar a desordem em sequências de dados. Para alcançar essa formulação, o autor utilizou um método analítico-dedutivo para propor propriedades axiomáticas gerais sobre medidas de desordem, introduzindo o conceito de algoritmo ótimo em relação a uma medida específica e tomando o número de inversões, $Inv(X)$, como métrica central. Como resultado, o estudo define otimalidade de forma relativa a cada medida e demonstra uma variante de *Insertion Sort* ótima em relação a três medidas naturais — o que não implica otimalidade do *Insertion Sort* clássico para a medida de inversões em geral. O trabalho estabelece assim fundamentos para a análise de algoritmos adaptativos. O trabalho foi apresentado em conferência (ICALP 1984) e publicado em versão estendida no *IEEE Transactions on Computers* [Mannila, 1985].

Em uma abordagem voltada a transformações preliminares, Hwang *et al*. [2000] investigaram o impacto de etapas de *presorting* sob a perspectiva do caso médio. Os autores empregaram uma metodologia analítica combinatória para modelar a variação na estatística de inversões após passadas iniciais de algoritmos como *Shellsort* e *Quicksort*, obtendo expressões em forma fechada para esperança e variância da redução de inversões e suas funções geradoras. Esse artigo não analisa o pré-processamento simétrico aqui proposto; nenhuma fórmula teórica deste trabalho é derivada dele. O valor de $45{,}19\%$ medido no Aleatório em $n{=}10^4$ é um resultado empírico (Tabela 3), sem derivação fechada neste artigo.

Estudos mais recentes aprofundam a interface entre adaptatividade e hardware. Edelkamp and Weiß [2016] propuseram o *BlockQuicksort*, que reorganiza o particionamento em blocos com *buffers* de índices para evitar desvios condicionais, obtendo aceleração substancial. Esse trabalho é citado apenas como contexto; sem contadores de hardware, este artigo não atribui causalmente nenhuma degradação medida a falhas de predição (Seção 3.5). Por sua vez, Auger *et al*. [2018] analisaram a complexidade de pior caso do *Timsort* com provas de indução e testes empíricos em sequências com blocos ordenados (*runs*), derivando limites assintóticos para o pior caso; garantias de estabilidade não são contribuição desse artigo.

Diferenciando-se das abordagens mapeadas na literatura, este trabalho objetiva avaliar uma heurística de Pré-processamento Simétrico projetada para a mitigação direta de inversões em tempo linear $O(n)$, investigando suas implicações teóricas e práticas. A metodologia desenvolvida estabelece um arcabouço experimental que integra a detecção de *runs* como *baseline* competitivo e submete a técnica proposta a ensaios empíricos em dez topologias de entrada com variações no grau de ordenação. Como contribuição e proposta de análise, a pesquisa delimita o escopo de aplicabilidade do pré-processamento ao isolar os cenários em que o ganho pela redução de inversões supera os custos de instrução e de predição de desvios, fornecendo uma caracterização sistemática do comportamento de algoritmos adaptativos e não adaptativos sob essa intervenção preliminar.

3 Métodos e Técnicas da Pesquisa

A presente pesquisa é classificada, quanto à sua natureza, como aplicada. Segundo Gil [2019], a pesquisa aplicada caracteriza-se pela geração de conhecimentos voltados à aplicação prática e dirigidos à solução de problemas específicos. Nesse contexto, o estudo avalia um pré-processamento simétrico de baixo custo, capaz de reduzir o impacto das inversões presentes nas entradas e de melhorar o desempenho de algoritmos de ordenação, contribuindo diretamente para a eficiência de sistemas computacionais.

Quanto à abordagem, a pesquisa é classificada como quantitativa. De acordo com Sampieri *et al*. [2021], a abordagem quantitativa fundamenta-se na coleta de dados baseada em medição numérica e em análise estatística, com o objetivo de testar hipóteses e estabelecer padrões de comportamento. Nesse sentido, o trabalho mensura o tempo de execução e a contagem de inversões de cada algoritmo, com e sem pré-processamento, por meio das métricas estatísticas fornecidas pela biblioteca *Criterion*.

Quanto aos objetivos, a pesquisa é classificada como descritiva, com caráter exploratório. Segundo Gil [2019], a pesquisa descritiva tem como preocupação central caracterizar fenômenos e mapear relações entre variáveis, sem pretensão de estabelecer causalidade — reserva necessária aqui, pois os mecanismos microarquiteturais (predição de desvios, cache) permanecem como hipóteses (Seção 3.5). Desse modo, o estudo busca descrever como a topologia da entrada (grau de desordem, presença de duplicados e padrões estruturais) e o pré-processamento simétrico se relacionam com o tempo de execução e a quantidade de inversões dos algoritmos avaliados.

Quanto aos procedimentos, a pesquisa é classificada como experimental. Conforme Gil [2019], o delineamento experimental consiste em determinar um objeto de estudo, selecionar as variáveis capazes de influenciá-lo e definir as formas de controle e de observação dos efeitos produzidos. Assim, o trabalho manipula de forma controlada a topologia e o tamanho das entradas (variáveis independentes) e observa o tempo de execução e as inversões (variáveis dependentes), sob semente fixa, geração pareada de vetores e ambiente computacional padronizado. As hipóteses orientadoras, formuladas em termos gerais e refinadas após a exploração inicial dos dados, são: **H1** — o pré-processamento reduz substancialmente I em entradas de alta desordem, com efeito dependente da geometria da desordem; **H2** — há ganho líquido de tempo quando a economia na ordenação supera C_{pre}, operacionalizada para o *Insertion Sort* como $\Delta I \gtrsim 5n$; **H3** — não há ganhos sistemáticos para métodos não adaptativos nem quasilineares. As qualificações específicas (Tubo e Runs invisíveis à varredura; *Merge Sort* em entradas alinhadas) emergiram da análise e são reportadas como resultados, não como predições *a priori*. Mecanismos microarquiteturais (predição de desvios, cache) são tratados como hipóteses, não como medições (Seção 3.5).

3.1 Configuração

Os experimentos foram realizados em ambiente controlado, cujas especificações de hardware, sistema operacional e ferramentas são apresentadas na Tabela 1. Para reduzir possíveis fontes de variabilidade, o plano de energia foi mantido em alto desempenho e os processos de segundo plano foram minimizados durante a execução dos testes. O código foi compilado em modo *release*, com nível máximo de otimização (`opt-level = 3`). A geração das entradas empregou uma semente fixa, garantindo reprodutibilidade e correspondência entre os conjuntos submetidos às diferentes condições experimentais.

Tabela 1. Ambiente experimental e ferramentas utilizadas.

Componente	Configuração / Especificação
Sistema operacional	Windows 11 Pro (build 10.0.26200)
Processador	AMD Ryzen 7 8700G (8 núcleos / 16 threads)
Memória	16 GB DDR5 5600MT/s (dual channel)
Linguagem	Rust 1.96.0 (perfil *release*)
Ferramentas de Medição	Criterion 0.5 e Ferramenta CLI do Projeto
Geração de dados	Seeds 42–46 (`ChaCha8Rng`), entradas pareadas

Para avaliar a resiliência e a adaptabilidade dos algoritmos sob diferentes graus de entropia, foram empregados dez padrões estruturais de vetores de entrada (Tabela 2). Invertido, Zigue-zague e Tubo foram desenhados pelos autores a partir da simetria da varredura proposta (Tartarugas e Serra são sintéticas), de modo que há viés de seleção embutido: Invertido e Zigue-zague são alinhados à simetria e tendem a favorecer a técnica, enquanto Tubo, embora construído sobre a mesma simetria, não apresenta inversões espelhadas e é praticamente insensível à varredura. A aleatoriedade determinística foi assegurada pelo pareamento das sementes de geração: para cada combinação de topologia e tamanho, um conjunto fixo de 50 vetores é gerado com a semente $42 \oplus n \oplus$ tipo (via `ChaCha8Rng`) e compartilhado por todos os algoritmos e pelas duas variantes (pura e com pré-processamento), garantindo comparabilidade pareada. A mistura por XOR pode, em princípio, colidir entre combinações distintas; nenhuma colisão afeta o pareamento dentro de cada célula, pois o mesmo *pool* é reusado pelos dois braços. Nas topologias determinísticas, as 50 réplicas (e os cinco *pools*) são cópias idênticas: o tamanho inferencial $R{=}50$ vale apenas para as topologias estocásticas, e o *bootstrap* nas determinísticas mede somente ruído temporal. A matriz central é replicada com cinco *pools* independentes (seeds 42–46) para análise de estabilidade entre *pools* (Seção 3.3). Topologias padrão da literatura (ex.: k-ordenado, perturbações locais, baterias de pdqsort/ipnsort) ficam como trabalho futuro.

Os tamanhos medidos são $n \in \{16, 32, 64, 128, 256, 512, 1.000, 5.000, 10.000, 100.000, 1.000.000, 10.000.000\}$, com gating por célula: quadráticos (*Insertion Sort*, *Bubble Sort*, *Selection Sort* e *DescReverse*) vão até $n{=}100.000$ (em $n{=}10^6$ cada execução levaria minutos); $n{=}10^7$ cobre apenas *Merge Sort*, *Quicksort* e baselines da biblioteca padrão, no regime fora de cache ($\approx$40 MB por vetor `i32`). A topologia Real é limitada ao tamanho nativo do asset ($N{=}477.104$, sem *tiling*) e portanto participa até $n{=}100.000$.

O custo isolado do pré-processamento (C_{pre}) foi adicionalmente caracterizado em $n \in$

$\{1.000, 5.000, 10.000, 20.000, 100.000, 1.000.000\}$ (Tabela 7) para melhor evidenciar a linearidade; o ponto n=20.000 foi coletado exclusivamente para essa curva de custo via ferramenta CLI (`seed` 42), sob mesmo protocolo de geração pareada, e não faz parte da matriz principal de comparação de tempos de ordenação.

Tabela 2. Conjuntos de dados e caracterização das topologias de entrada.

Topologia	Descrição do Padrão Estrutural
Aleatório	permutação uniforme de $[0, n-1]$ via `shuffle` com `ChaCha8Rng`
Tartarugas	$a[i] = i+n$ se $i < n/2$, senão $a[i] = i \bmod 10$
Zigue-zague	$a[i] = i$ se i par, senão $a[i] = n - i$; determinístico
Quase ordenado	$[0, n-1]$ ordenado com $n/100$ trocas de pares aleatórios
Duplicados	$a[i] \sim \mathcal{U}\{0, 1, 2\}$ i.i.d.
Invertido	$a[i] = n - i$ (estritamente decrescente); determinístico
Serra	$a[i] = i \bmod 16$
Tubo	$a[i] = i$ se $i < n/2$, senão $a[i] = n - 1 - i$
Runs	4 blocos invertidos disjuntos nos quartis do vetor
Real	prefixo de n bytes do asset congelado v1: 40 arquivos-fonte do próprio repositório no *commit* `c3b0bbe` (SHA-256 em `dados_reais/README.md`; N=477.104, sem *tiling*); cada byte vira um `i32` (faixa 0–255, muitos duplicados); rótulo operacional, sem representatividade geral

3.2 Especificação e Implementação dos Algoritmos Avaliados

Para assegurar a validade interna dos experimentos, a estabilidade das medições de tempo e a reprodutibilidade dos resultados, todos os algoritmos de ordenação foram implementados na linguagem Rust (versão 1.96.0), operando diretamente sobre fatias mutáveis de inteiros de 32 bits (`&mut [i32]`). As implementações evitaram dependências externas de abstrações de alto nível, padronizando a manipulação de memória e adotando otimizações idiomáticas no nível de instrução.

3.2.1 Algoritmos Quadráticos

- **Bubble Sort:** Instanciado em sua versão adaptativa otimizada. O algoritmo realiza varreduras sucessivas reduzindo o limite superior a cada iteração ($n - 1 - i$). A adaptabilidade é garantida por uma flag booleana de controle (`trocou`) que monitora a ocorrência de trocas em cada passada; caso uma varredura seja concluída sem nenhuma permuta de elementos, a execução é interrompida precocemente, garantindo complexidade linear $O(n)$ em vetores previamente ordenados.
- **Insertion Sort:** Implementado conforme a mecânica tradicional de inserção direta por deslocamento. Para cada elemento da sub-fatia não ordenada, armazena-se o valor corrente em uma variável temporária (`valor_atual`) e promovem-se deslocamentos à direita dos elementos antecedentes estritamente maiores. A busca e o deslocamento encerram-se assim que a posição correta é encontrada ou ao atingir o limite esquerdo ($j = 0$), momento em que o valor temporário é atribuído. Essa abordagem minimiza o *overhead* de escrita em relação às implementações baseadas em trocas sucessivas (*swaps*).
- **Selection Sort:** Implementado em sua forma clássica não adaptativa e *in-place*. Em cada iteração i do laço externo, o algoritmo varre o segmento não ordenado $[i + 1, n - 1]$ para identificar o índice do menor elemento (`indice_minimo`). A troca (`swap`) com a posição i é executada condicionalmente apenas se o menor elemento encontrado residir em um índice distinto (`indice_minimo` $\neq i$). O algoritmo mantém um custo de comparações estrito de $O(n^2)$, independentemente da ordenação prévia da entrada.

3.2.2 Algoritmos Quasilineares

- **Merge Sort:** Desenvolvido segundo a estratégia *top-down* (divisão e conquista) com garantia de ordenação estável. Para eliminar o *overhead* de alocação dinâmica contínua na pilha recursiva — o que distorceria as medições temporais —, a implementação aloca um único vetor auxiliar temporário de tamanho n (`vec![0; len]`) no ponto de entrada da função. Esse *buffer* é passado por referência mutável através das chamadas recursivas. Os pontos médios das sub-fatias são calculados por meio do deslocamento de bits à direita ($(\text{fim} - \text{inicio}) \gg 1$), evitando operações de divisão inteira. A fase de intercalação (`intercalar`) copia os dados da fatia atual para o *buffer* auxiliar e reconstrói o segmento ordenado no vetor original operando com três ponteiros de índice.
- **Quicksort:** O método original é de Hoare [1962]. Aqui, foi implementado de forma recursiva utilizando particionamento de três vias (*Dutch National Flag* / algoritmo de Dijkstra) aliado à seleção de pivô por mediana de três. O pivô é escolhido como a mediana entre os elementos nos índices inicial, central e final do subvetor. O particionamento de três vias rearranja o subvetor em três regiões contíguas: elementos menores que o pivô, elementos iguais ao pivô e elementos maiores que o pivô, mantendo índices `lt`, `i` e `gt` para delimitar as fronteiras. Essa estratégia confere tempo médio $O(n \log n)$ e robustez a muitas chaves duplicadas (sem garantia de pior caso: a mediana de três não exclui entradas adversariais). A recursão é aplicada apenas às regiões de elementos menores (`[inicio, lt-1]`) e maiores (`[gt+1, fim]`) que o pivô.

3.2.3 Baselines Competitivos

- `sort_unstable`: a ordenação instável da biblioteca padrão do Rust (implementação *ipnsort* nas versões recentes [Rust Standard Library, 2026]), usada como referência competitiva geral; métodos especializados (ex.: *radix sort* para `i32`) estão fora do escopo.
- `sort`: a ordenação estável da biblioteca padrão (que detecta *runs* ordenados), incluída como referência estável.
- `desc_reverse`: verificação $O(n)$ de *run* estritamente decrescente; se a entrada é estritamente decrescente, ela é revertida *in-place* e então executa-se o *Insertion Sort* (caso contrário, *Insertion Sort* direto). Testa se uma detecção trivial de *run* explica os ganhos no Invertido.

Em todas as implementações, verificações preliminares de contorno (*early returns*) asseguram que fatias com tamanho

$n < 2$ retornem imediatamente sem incorrer em chamadas de função ou alocações desnecessárias.

3.3 Protocolo Experimental e Raciocínio de Cálculo de Inversões

O protocolo de medição empírica foi desenhado para isolar interferências do ambiente e garantir a reprodutibilidade estatística das métricas temporais. A quantificação do tempo de execução utilizou o arcabouço *Criterion.rs*, que executa ciclos automatizados de aquecimento de memória cache (*warm-up*) e realiza reamostragens estatísticas (*bootstrapping*) para estimar intervalos de confiança de 95% (`confidence_level=0.95`, `significance_level=0.05`). Para cada combinação de algoritmo, topologia e tamanho de vetor n, foram calculados o tempo médio, a mediana e os intervalos de confiança de 95%.

A configuração de medição foi ajustada por perfil (valores configurados; o Criterion pode estender a medição até estabilizar): 50 amostras com aquecimento de 2 s e janela de 5 s na maioria dos cenários; 30 amostras com aquecimento de 1 s e janela de 5 s para quadráticos em n=100.000; 50 amostras com aquecimento de 2 s e janela de 10 s para quasilineares em $n \geq 10^6$. Cada iteração do Criterion ordena exatamente um vetor: o *setup* (`iter_batched`) clona `pool[i]` fora da região cronometrada (fora do temporizador) e a rotina cronometrada executa `medir_puro` ou `medir_com_pre` sobre o clone, com `black_box` no resultado para impedir eliminação de código morto. O tempo "por vetor" é o tempo por iteração agregado pelo Criterion em amostras. Em pseudocódigo, por célula: `pool = gerar(seed, tipo, n); validar fora do timing`; para cada braço, `bench{ vetor = clone(pool[i++ mod 50]); t = medir(vetor) }`. A variante com pré-processamento mede o custo composto $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$, e um braço separado mede somente C_{pre} no mesmo *pool*. Validação de corretude e contagem de inversões ocorrem fora da região cronometrada.

Critério estatístico: para cada célula e cada *pool*, as variantes pura e com pré-processamento são comparadas nos *mesmos* vetores (delineamento pareado, R=50 posições) por um experimento pareado dedicado, que alterna a ordem dos braços entre repetições para controlar deriva temporal. A estatística de efeito é a média das log-razões por vetor $\overline{\log(T_{\text{pre}}/T_{\text{puro}})}$, com intervalo de confiança percentílico por *bootstrap* de 95% (B=10.000 reamostragens, RNG determinística por célula). Um efeito é relatado como significativo quando o IC pareado exclui o zero (* no material suplementar); † marca efeitos significativos *e* estáveis em sinal nos cinco *pools*. Adota-se ainda um limiar prático de $\pm 2\%$: efeitos significativos abaixo desse módulo são relatados como irrelevantes na prática (ex.: *Merge Sort*/Real e *Quicksort*/Tubo em n=10^6). Com centenas de células testadas, há risco de falsos positivos em efeitos pequenos; a exigência de estabilidade nos cinco *pools* mitiga, mas não elimina, o problema de comparações múltiplas. Células significativas no pareamento mas instáveis entre *pools* (ex. *Quicksort*/Quase ordenado em n=10^6) são sinalizadas como não robustas; *Selection Sort*/Zigue-zague em n=10.000 é não significativo no pareamento.

Além das métricas temporais, a corretude de cada cenário foi verificada fora da região cronometrada: para todo (algoritmo, topologia, tamanho), confirma-se que a saída é uma permutação ordenada da entrada (comparação com `sort_unstable`) tanto na variante pura quanto na com pré-processamento. O número de inversões de cada vetor do conjunto compartilhado, antes e depois do pré-processamento, é registrado em arquivos auxiliares (*companion*) do harness, garantindo que as tabelas de inversões deste artigo derivam exatamente dos mesmos vetores cronometrados.

A quantificação experimental do número exato de inversões I utiliza uma implementação baseada no algoritmo MergeSort modificado, com complexidade temporal $O(n \log n)$ [Cormen *et al.*, 2022]. Durante a etapa de intercalação de dois subvetores ordenados L e R, quando um elemento $R[j]$ é menor que $L[i]$, adiciona-se exatamente $|L| - i$ ao contador de inversões. Essa abordagem elimina o gargalo quadrático da contagem direta $O(n^2)$, permitindo a avaliação de vetores com ordens de grandeza de 10^5 a 10^6 elementos dentro do protocolo experimental.

3.4 Formalização do Pré-Processamento Simétrico

O procedimento de pré-processamento simétrico proposto opera como uma etapa determinística de varredura única sobre o vetor de entrada A de tamanho n. O objetivo desta etapa é reestruturar a disposição dos elementos através de trocas espelhadas e ajustes locais, antecedendo a execução dos algoritmos de ordenação avaliados. A estrutura procedural e as regras de controle do algoritmo estão formalizadas no Algoritmo 1.

Algoritmo 1: Algoritmo de Pré-Processamento Simétrico

```
Input: Vetor A de tamanho n
Output: Vetor A parcialmente reorganizado
n ← tamanho de A;
if n < 2 then
    return;
end
ultimo ← n − 1;
meio ← ⌊n/2⌋;
for i ← 0 to meio − 1 do
    j ← ultimo − i;
    if A[i] > A[j] then
        trocar(A[i], A[j]);
    end
    if i + 1 < meio then
        if A[i] > A[i + 1] then
            trocar(A[i], A[i + 1]);
        end
    end
    if j − 1 > meio then
        if A[j] < A[j − 1] then
            trocar(A[j], A[j − 1]);
        end
    end
end
```

O algoritmo itera sobre a primeira metade do vetor, varrendo os índices $i \in [0, \lfloor n/2 \rfloor - 1]$. A cada iteração, calcula-se o índice simétrico correspondente no extremo oposto do

vetor, dado por $j = (n-1) - i$. A execução do laço é composta por três verificações condicionais estruturadas:

1. **Troca Simétrica Extrema:** Avalia-se a relação de ordem entre as extremidades opostas. Se $A[i] > A[j]$, efetua-se a troca direta dos elementos, posicionando o menor elemento no segmento esquerdo e o maior no segmento direito.
2. **Ajuste Adjacente Esquerdo:** Para os índices em que $(i+1) < \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade esquerda. Se $A[i] > A[i+1]$, realiza-se a troca entre os elementos adjacentes.
3. **Ajuste Adjacente Direito:** Para os índices em que $(j-1) > \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade direita. Se $A[j] < A[j-1]$, executa-se a troca entre os elementos adjacentes.

As condições de contorno garantem que as trocas adjacentes operem estritamente dentro de seus respectivos subdomínios (esquerdo e direito), sem ultrapassar o ponto médio do vetor. Em termos de complexidade computacional, o algoritmo executa exatamente $\lfloor n/2 \rfloor$ iterações. Em cada passo do laço, realiza-se um número constante de comparações (no máximo 3) e trocas (no máximo 3), resultando em uma complexidade temporal de pior caso $T(n) = O(n)$. Como a reestruturação dos dados é realizada *in-place* no próprio vetor A, a complexidade espacial auxiliar é estritamente $O(1)$.

Uma propriedade útil de monotonicidade é que trocar um par espelhado invertido nunca aumenta I: remove-se a inversão do par (i, j) e cada posição intermediária k com $a_j < a_k < a_i$ contribui com redução líquida, de modo que a passada é um passo de descida em I. Apenas para o passo espelhado isolado (sem os ajustes adjacentes) e sob independência uniforme, uma conta de primeira ordem dá $\mathbb{E}[\Delta I] = n^2/12 + O(n)$, cerca de um terço da base $\mathbb{E}[I] = n^2/4$. Essa conta não é uma derivação do algoritmo completo: o valor medido no Aleatório ($\Delta I \approx 11{,}3$M em $n{=}10^4$, i.e. $45{,}19\%$) supera a previsão ($\approx 8{,}3$M) por $\approx 2{,}9$M, diferença que os ajustes adjacentes não explicam sozinhos (cada troca adjacente remove exatamente 1 inversão, no máximo $\approx n$ no total). Uma hipótese, não verificada, é que o ajuste adjacente desloca o maior valor para a direita (e o menor para a esquerda) antes da comparação espelhada seguinte, violando a independência assumida e elevando $P(A[i] > A[j])$ acima de $1/2$. Uma ablação do passo espelhado isolado (mesmo laço sem os ajustes adjacentes), em réplica independente com permutações uniformes, mediu ΔI médio de 8,36M em $n{=}10^4$ (33,43% de I), contra $n^2/12 \approx 8{,}33$M previstos, enquanto o algoritmo completo atingiu 11,32M (45,23%), reproduzindo os $45{,}19\%$ do *pool* principal (padrão confirmado em $n{=}10^3$ e $5{\times}10^3$; detalhes no material suplementar). A conta de primeira ordem descreve, portanto, o passo isolado; o excedente de ≈2,9M decorre da interação entre ajustes adjacentes e comparações espelhadas seguintes, não dos ajustes isolados ($\leq n$). A forma fechada desse acoplamento permanece como trabalho futuro.

Para o *Insertion Sort*, cujo custo é de $\approx c$ por inversão ($c \approx 0{,}21$ ns medidos, Seção 4.3), o pré-processamento compensa se e somente se $C_{\text{pre}} < c\,\Delta I$. Com $C_{\text{pre}} \approx \alpha n$ ($\alpha \approx 1$ ns/elemento no geral, 0,5–3,0 ns por topologia, Tabela 8), a regra prática, restrita ao *Insertion Sort*, é $\Delta I \gtrsim 5n$ inversões removidas; os *breakevens* por topologia seguem da Tabela 8 (ex. 144.000 para Aleatório, 28.000 para Tubo em $n{=}10.000$). Cada desfecho do *Insertion Sort* em $n{=}10.000$ nas Tabelas 4–6 obedece a essa regra: Aleatório ($\Delta I{=}11{,}3$M), Serra (12,5M) e Duplicados (10,3M) ganham, enquanto Tubo (10.000) e Runs (5.000) ficam abaixo do *breakeven* e são neutros. A regra não se estende ao *Bubble Sort*, cujo tempo não acompanha I (Seção 4.3). O procedimento não preserva estabilidade (trocas espelhadas ultrapassam chaves iguais), limitando o uso com registros cuja ordem de iguais importe.

3.5 Ameaças à Validade

Internas: máquina, SO (Windows 11), compilador (Rust 1.96.0, *release*, `opt-level=3`) e dados `i32` únicos; frequência da CPU, afinidade SMT e resolução do temporizador não foram controlados além do plano de alto desempenho. *De construção:* C_{pre} é medido por topologia no mesmo harness (Tabela 8); apenas chaves `i32` e uma implementação por algoritmo foram testadas; sem ablação sistemática dos componentes da varredura em todas as topologias (a ablação do passo espelhado isolado restringe-se a permutações uniformes; Seção 3.4). *Externas:* topologias em parte autorais (viés de seleção), pseudo-réplicas determinísticas, quadráticos limitados a $n{\leq}10^5$, Duplicados fixos em $k{=}3$, rótulo "Real" operacional; $n{=}10^6$ (≈4 MB) reside em cache no 8700G e $n{=}10^7$ exercita o regime fora de cache. *Estatísticas: bootstrap* pareado com estabilidade entre *pools* para $n{=}10^4$ e 10^6; demais tamanhos usam os ICs do Criterion. *De medição:* sem contadores de hardware; atribuições a preditor de desvios e cache são hipóteses, assim como a explicação do excedente sobre $n^2/12$ e o mecanismo dos ganhos do *Merge Sort*. A reprodução dos experimentos pode ser realizada via `cargo bench` e pelas rotinas disponibilizadas no repositório da pesquisa (Seção 7).

4 Resultados e Discussão

Esta seção apresenta e discute os resultados: redução de inversões e viés geométrico das entradas, tempo total de ordenação nos métodos quadráticos e quasilineares, e os limites de aplicabilidade da técnica.

4.1 Redução de Inversões e Viés Estrutural das Entradas

A Figura 1 ilustra a variação das inversões observadas nos vetores de 10.000 elementos, cujos valores quantitativos exatos e porcentagens de redução estão detalhados na Tabela 3. Nas entradas não alinhadas com desordem não estruturada (Aleatório, Tartarugas, Quase ordenado, Duplicados, Real e Serra), o pré-processamento remove de 40% a 62% das inversões em $n{=}10^4$ — uma redução absoluta da ordem de 10^7 nesse tamanho (série de n não reportada para inversões; sem afirmação assintótica). Duas topologias não alinhadas resistem quase por completo à varredura: *Tubo* (0,04%) e *Runs* (0,16%), cuja geometria de desordem as comparações espelhadas jamais alcançam. Contudo, a interpretação da eliminação total das inversões (100%) no cenário *Invertido* exige sobriedade teórica e analítica.

[*] 99,99% (2.501 inversões restantes); [**] 100,00% exato (zero inversões restantes). A diferença de arredondamento im-

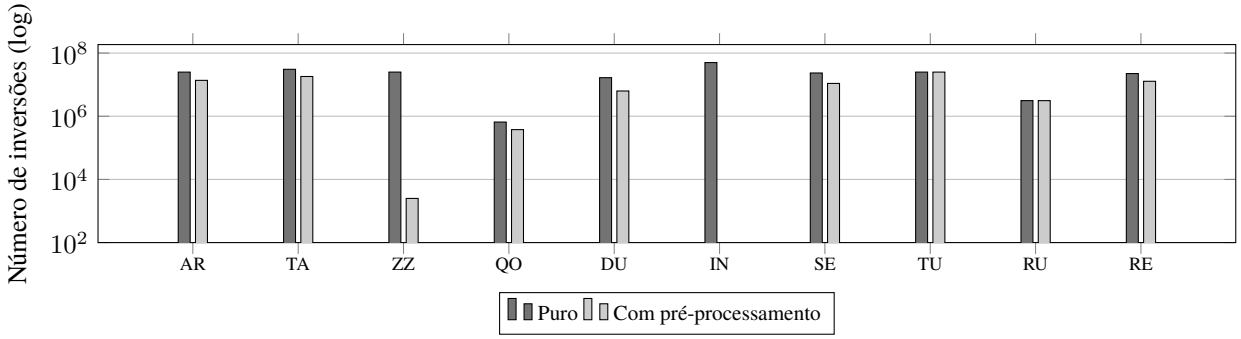


Figura 1. Número de inversões antes e após o pré-processamento simétrico para vetores de 10.000 elementos (escala logarítmica; no cenário Invertido o valor pós-pré-processamento é zero e não aparece na escala log). As abreviações são: AR (Aleatório), TA (Tartarugas), ZZ (Zigue-zague), QO (Quase ordenado), DU (Duplicados), IN (Invertido), SE (Serra), TU (Tubo), RU (Runs) e RE (Real).

Tabela 3. Redução de inversões proporcionada pelo pré-processamento simétrico em vetores de 10.000 elementos (média dos 50 vetores do *pool* pareado; mesmos vetores cronometrados).

Tipo	Inversões	Pós-pré	Redução
Aleatório	24.960.120	13.681.366	45,19%
Tartarugas	30.613.750	18.133.750	40,77%
Zigue-zague	24.997.500	2.501	99,99%*
Quase ordenado	654.922	377.149	42,41%
Duplicados	16.633.993	6.300.346	62,12%
Invertido	49.995.000	0	100,00%**
Serra	23.400.000	10.937.472	53,26%
Tubo	24.995.000	24.985.004	0,04%
Runs	3.122.500	3.117.505	0,16%
Real	22.447.037	12.789.942	43,02%

pede interpretação de equivalência entre os dois casos. Tubo (0,04%) e Runs (0,16%) são estruturalmente invisíveis à varredura espelhada (Seção 4.1).

Como o Algoritmo 1 executa a troca direta entre $a[i]$ e $a[n-1-i]$, um vetor estritamente invertido (onde $a[i] > a[n-1-i]$ para todo $i < n/2$) sofre uma reversão geométrica completa da sua sequência em $O(n)$ operações. A resolução do caso Invertido decorre da geometria da transformação do pré-processamento, configurando um caso ideal onde a topologia da desordem coincide perfeitamente com a simetria aplicada.

O comportamento observado no cenário *Zigue-zague* também pode ser interpretado à luz dessa propriedade estrutural do pré-processamento. Embora sua topologia não corresponda a uma inversão global como no caso *Invertido*, a alternância sistemática entre blocos com orientações distintas produz relações de desordem que são parcialmente capturadas pelas comparações entre posições simétricas. Dessa forma, a troca direta entre $a[i]$ e $a[n-1-i]$ corrige simultaneamente um grande número de relações fora de ordem, reduzindo as inversões de 24.997.500 para apenas 2.501, uma redução de aproximadamente 100% (99,99%).

O resultado evidencia que a eficácia do pré-processamento não se restringe ao caso de reversão completa: ela também se manifesta em topologias cuja estrutura apresenta forte alinhamento com a simetria utilizada pela transformação, ainda que de maneira parcial. Essa característica ajuda a explicar os ganhos particularmente elevados observados posteriormente nos algoritmos de Insertion Sort e Bubble Sort para essa entrada.

4.2 Impacto no Tempo de Execução e Custo do Pré-processamento

A Tabela 4 resume as faixas de ganho obtidas sob vetores de 10.000 elementos (valores por célula no material suplementar), evidenciando como a eficiência do pré-processamento depende da adaptabilidade de cada algoritmo. Em entradas não alinhadas, os métodos quadráticos adaptativos obtiveram ganhos expressivos: o *Insertion Sort* avançou de 28,6% a 60,3% (Aleatório $+43{,}9\%$) e o *Bubble Sort* de 14,9% a 26,2%. Os maiores benefícios concentraram-se nas entradas alinhadas à varredura: no cenário *Zigue-zague*, o tempo do *Insertion Sort* caiu de $5.249{,}97\,\mu s$ para $13{,}60\,\mu s$ ($\times 386$) e o do *Bubble Sort* de $18.469{,}42\,\mu s$ para $16{,}00\,\mu s$ ($\times 1.155$); no *Invertido*, os *speedups* alcançaram $\times 837$ e $\times 2.280$, respectivamente ($+99{,}9\%$ e $+100{,}0\%$).

Em contrapartida, algoritmos de complexidade $O(n \log n)$, como o *Merge Sort*, apresentaram resultados mistos em $n = 10.000$ (decomposição de $-14{,}2\%$ a $+15{,}7\%$; e até $+18{,}6\%$ em $n = 10^6$, Tabela 9), uma vez que o custo da etapa prévia tende a superar eventuais ganhos em distribuições sem estrutura favorável. Sob a validação por *bootstrap* pareado (Seção 3.3), não apresentaram diferença estatística significativa em $n = 10.000$ as combinações Merge/Duplicados ($-1{,}2\%$) e Selection para a maioria das entradas não alinhadas. Ademais, ressalta-se que Selection/Tartarugas ($+3{,}1\%$) mostrou-se instável entre *pools* e Quicksort/Quase ordenado ($-4{,}5\%$) não atingiu significância no pareamento.

Tabela 4. Faixa de ganho do pré-processamento por algoritmo para vetores de 10.000 elementos (mínimo e máximo sobre as seis topologias clássicas; tipos entre parênteses). Os valores por célula, com marcas de significância, estão no material suplementar.

Algoritmo	Ganho mínimo	Ganho máximo
Merge	$-14{,}2\%$ (Tartarugas)	$+15{,}7\%$ (Zigue-zague)
Quick	$-38{,}4\%$ (Invertido)	$+0{,}5\%$ (Zigue-zague)
Insertion	$+28{,}6\%$ (Quase ordenado)	$+99{,}9\%$ (Invertido)
Bubble	$+14{,}9\%$ (Aleatório)	$+100{,}0\%$ (Invertido)
Selection	$-9{,}1\%$ (Invertido)	$+3{,}1\%$ (Tartarugas)

O *Insertion Sort* com pré-processamento em entrada Aleatória ($2.921{,}09\,\mu s$) permanece $\approx 8{,}1\times$ mais lento que o *Quicksort* puro ($360{,}74\,\mu s$), e o próprio `sort_unstable` ($80{,}41\,\mu s$) é $\approx 36\times$ mais rápido que o melhor híbrido na mesma entrada. Há uma exceção: no Zigue-zague em

Tabela 5. Baselines competitivos: tempo médio por execução (em µs, média ± meia-largura do IC 95%) e ganho do pré-processamento para vetores de 10.000 elementos. `sort_unstable` e `sort` são os ordenadores da biblioteca padrão do Rust; `desc_reverse` é o Insertion Sort precedido de detecção $O(n)$ de run descendente com reversão. Marcas de significância conforme a Seção 3.3.

Algoritmo	Tipo	Puro	Com pré	Ganho
`sort_unstable`	Aleatório	80,41 ± 2,35	107,55 ± 0,48	−33,8%†
`sort_unstable`	Tartarugas	47,26 ± 0,28	51,34 ± 0,28	−8,7%†
`sort_unstable`	Zigue-zague	70,63 ± 0,46	66,09 ± 0,29	+6,4%†
`sort_unstable`	Quase ordenado	76,10 ± 0,36	83,16 ± 0,43	−9,3%†
`sort_unstable`	Duplicados	15,08 ± 0,28	40,83 ± 0,19	−170,8%†
`sort_unstable`	Invertido	3,75 ± 0,12	7,86 ± 0,19	−109,7%†
`sort`	Aleatório	112,00 ± 0,69	140,99 ± 0,82	−25,9%†
`sort`	Tartarugas	23,44 ± 0,25	27,79 ± 0,17	−18,5%†
`sort`	Zigue-zague	93,45 ± 0,70	47,60 ± 0,29	+49,1%†
`sort`	Quase ordenado	124,33 ± 0,64	126,66 ± 0,47	−1,9%
`sort`	Duplicados	17,85 ± 0,55	43,79 ± 0,28	−145,3%†
`sort`	Invertido	4,48 ± 0,17	8,43 ± 0,22	−88,2%†
`desc_reverse`	Aleatório	5.297,50 ± 27,50	2.966,86 ± 17,17	+44,0%†
`desc_reverse`	Tartarugas	6.421,62 ± 30,47	3.826,81 ± 19,40	+40,4%†
`desc_reverse`	Zigue-zague	5.299,78 ± 31,93	13,42 ± 0,12	+99,8%†
`desc_reverse`	Quase ordenado	174,43 ± 3,28	126,71 ± 2,56	+27,4%†
`desc_reverse`	Duplicados	3.569,32 ± 17,29	1.414,40 ± 8,50	+60,4%†
`desc_reverse`	Invertido	11,03 ± 0,32	11,86 ± 0,21	−7,5%†

n=10^4, topologia construída em torno da simetria da varredura, o *Insertion Sort* com pré-processamento (13,60 μs) e o *Bubble Sort* com pré-processamento (16,00 μs) superam o `sort_unstable` puro (70,63 μs) e o `sort` puro (93,45 μs). No Invertido, o `desc_reverse` puro (11,03 μs) iguala o *Insertion Sort* com pré-processamento (12,81 μs): a detecção trivial de *run* captura sozinha todo o ganho *no Invertido*. No Zigue-zague, ao contrário, o `desc_reverse` puro (5.299,78 μs) fica muito acima dos 13,42 μs com pré-processamento, de modo que a afirmação não se estende às entradas alinhadas em geral. Os *speedups* extremos são propriedade da geometria dessas entradas, não do pré-processamento em geral.

Tabela 6. Faixa de ganho do pré-processamento por método nas topologias adicionais para vetores de 10.000 elementos (mínimo e máximo sobre Serra, Tubo, Runs e Real). Os valores por célula, com marcas de significância, estão no material suplementar.

Método	Ganho mínimo	Ganho máximo
Merge Sort	−5,2% (Runs)	+0,0% (Real)
Quicksort	−34,2% (Serra)	−1,0% (Tubo)
Insertion Sort	+0,0% (Tubo)	+52,3% (Serra)
Bubble Sort	−0,7% (Tubo)	+33,8% (Serra)
Selection Sort	−0,4% (Serra)	+1,2% (Runs)
`sort_unstable`	−26,8% (Real)	−6,3% (Tubo)
`sort`	−71,3% (Tubo)	−12,8% (Serra)
`desc_reverse`	−0,9% (Tubo)	+51,7% (Serra)

A Serra comporta-se como as topologias não estruturadas (Insertion +52,3%, Bubble +33,8%, `desc_reverse` +51,7%), o Real acompanha de perto o Aleatório (Insertion +42,2%, Bubble +13,6%), enquanto Tubo e Runs são neutros para os métodos clássicos (−5,2% a +2,5%, majoritariamente não significativos) — consistente com sua redução de inversões de ≈0%. Os ordenadores padrão degradam sob pré-processamento nas topologias adicionais (até −71,3% para `sort` no Tubo), pois o *overhead* $O(n)$ não lhes traz benefício.

A viabilidade prática dessa abordagem fundamenta-se no baixo custo computacional da etapa simétrica. Conforme a Tabela 7, o tempo de execução do pré-processamento é estritamente linear $O(n)$, com tempo unitário entre 0,95 e 1,15 ns por elemento (mínimo em 10^6), indicando ausência de gargalos de localidade mesmo em escala: apenas 11,51 μs para 10.000 elementos, amplamente compensado pela economia de milissegundos na ordenação adaptativa.

Tabela 7. Custo médio do pré-processamento simétrico em função do tamanho do vetor (média sobre os tipos disponíveis, via ferramenta CLI, seed 42; Real apenas até N=477.104, logo ausente em 10^6). O ponto n=20.000 foi incluído exclusivamente para caracterização da linearidade e não faz parte da matriz principal de tempos de ordenação. A média CLI difere da média do *harness* Criterion (Tabela 8) por metodologia e cobertura distintas.

Tamanho	Pré (µs)	ns/elemento
1.000	1,13	1,13
5.000	5,50	1,10
10.000	11,51	1,15
20.000	23,09	1,15
100.000	113,58	1,14
1.000.000	946,36	0,95

Tabela 8. Custo isolado do pré-processamento C_{pre} por topologia (mesmo harness Criterion da matriz de tempos; média, μs). A média CLI da Tabela 7 esconde forte heterogeneidade: entradas não estruturadas (Aleatório, Duplicados) custam ≈5× mais que as estruturadas em n=10.000. Real não tem ponto em 10^6 (teto do asset nativo).

Tipo	C_{pre} em 10^4	C_{pre} em 10^6
Aleatório	30,36	3.049,14
Tartarugas	5,66	537,28
Zigue-zague	5,72	545,30
Quase ordenado	7,31	713,73
Duplicados	27,63	2.668,85
Invertido	5,69	517,05
Serra	5,10	490,93
Tubo	5,86	589,98
Runs	5,49	558,90
Real	6,97	—

A heterogeneidade é em si informativa: as condições de troca da varredura são desvios dependentes dos dados; entradas Aleatórias e com Duplicados custam ≈30 e 28 μs em n=10.000, contra ≈5–7 μs nas estruturadas. Sem contadores de hardware, se essa diferença envolve pressão sobre o preditor de desvios permanece hipótese (Seção 3.5). Consequentemente, o *overhead* por célula deve ser lido na Tabela 8, não na média da Tabela 7: por exemplo, Merge/Aleatório em n=10.000 fica +11,46 μs mais lento contra um C_{pre} topologia-específico de 30,36 μs — ou seja, a fase de ordenação em si melhorou ligeiramente e a degradação líquida é pura aritmética de *overhead*.

4.3 Tempo Total de Ordenação e Análise Individual dos Algoritmos

A avaliação do tempo total de execução revela uma dicotomia no impacto do pré-processamento. No *Insertion Sort*, a eliminação prévia das inversões proporciona ganhos expressivos quando a redução absoluta ΔI é grande (28,6% a 100,0% em n=10.000), pois a varredura linear de baixo custo (≈ 1 ns/elemento) é amplamente compensada pelo trabalho local suprimido. Mecanisticamente, o tempo do *In-*

sertion Sort por inversão mostra-se notavelmente constante ($\approx 0{,}21$ ns/inversão em $n{=}10.000$ para a maioria das entradas), de modo que seus ganhos acompanham a redução de ΔI de forma direta, desviando apenas em entradas quase ordenadas ($0{,}29$ ns/inversão) pela dominância dos custos fixos sob pequeno volume de inversões.

Por outro lado, o tempo de execução do *Bubble Sort* permanece praticamente constante (16–23 ms) em entradas cujo volume de inversões varia por duas ordens de grandeza, revelando que seu desempenho é determinado pelo deslocamento máximo dos elementos à esquerda, e não por I (a regra $\Delta I \gtrsim 5n$ não se aplica a ele). Esse comportamento explica seu ganho modesto em distribuições aleatórias ($+14{,}9\%$ apesar da queda de 45% nas inversões) em contraste com a redução a uma única passada em entradas alinhadas (*speedups* de $\times 1.155$ a $\times 2.280$). A avaliação nas topologias adicionais revalida essa dinâmica: a topologia *Serra* passa a apresentar comportamento semelhante ao das entradas alinhadas (ganhos de $+52{,}3\%$ no Insertion e $+33{,}8\%$ no Bubble), enquanto *Tubo* e *Runs* — onde a varredura não afeta a desordem ($\approx 0\%$ de redução) — mantêm o desempenho neutro ($-0{,}7\%$ a $+2{,}5\%$).

O *Merge Sort* exibe resultados mistos segundo o critério pareado. O ganho concentra-se nas topologias em que o pré-processamento elimina a desordem estrutural: *Zigue-zague* ($+15{,}7\%$) e *Invertido* ($+3{,}3\%$) em $n{=}10.000$, crescendo para $+18{,}6\%$ e $+6{,}0\%$ em $n{=}10^6$ (Tabela 9). Em *Aleatório*, *Tartarugas* e *Quase ordenado*, porém, o total degrada de forma significativa. Essas degradações são aritméticas, como mostra a decomposição por topologia da Tabela 8, e não uma perturbação da distribuição de chaves. O mecanismo dos ganhos em Zigue-zague e Invertido fica como hipótese: maior previsibilidade dos desvios na intercalação, e não as inversões por si, com teste dedicado em trabalho futuro.

O *Quicksort* aqui implementado — três vias de Dijkstra com mediana de três, sem as proteções das implementações de referência [Bentley and McIlroy, 1993; Musser, 1997] — degrada sistematicamente sob pré-processamento ($-4{,}5\%$ a $-8{,}0\%$ na maioria das topologias; $-34{,}4\%$ e $-38{,}4\%$ em *Duplicados* e *Invertido*). A base já é lenta em entradas estruturadas mesmo sem pré-processamento (até 4.070 ms no Tubo em $n{=}10^6$, contra $\approx$53 ms no Aleatório), comportamento esperado dessa combinação pivô/particionamento; os resultados refletem a implementação avaliada, e a validação contra pdqsort fica como trabalho futuro. Grande parte da degradação está na escala do próprio *overhead* (ex. $+14{,}25\,\mu s$ em *Duplicados* a $n{=}10.000$, ante C_{pre} de $27{,}63\,\mu s$).

No *Invertido*, a entrada pré-processada ordena muito mais devagar ($368{,}64$ ms em $n{=}10^6$) que a aleatória ($54{,}67$ ms): o *harness* instrumentado (material suplementar) indica particionamento degenerado, com a mediana de três escolhendo pivôs quase extremos nos subintervalos (profundidade rasa). O gatilho exato permanece como trabalho futuro; a degradação delimita-se a esta implementação.

Assim, para a classe $O(n \log n)$, a técnica não se mostra benéfica na implementação avaliada, com exceções pontuais (Merge/Zigue-zague e Merge/Invertido em ambas as escalas). Quicksort/Quase ordenado, a única exceção aparente em larga escala ($+5{,}5\%$ em $n{=}10^6$, após $-4{,}5\%$ em $n{=}10.000$ e $-9{,}3\%$ em $n{=}10^5$), troca de sinal entre *pools* na análise pareada e é, portanto, relatado como não robusto em vez de ganho. Esses resultados delimitam a aplicabilidade do pré-processamento, indicando que a redução de inversões, isoladamente, não garante ganhos para algoritmos dessa classe.

O comportamento do *Selection Sort* confirma sua natureza não adaptativa: como o número de comparações é fixo ($n(n-1)/2$) independentemente da configuração inicial, a redução das inversões não se traduz em ganho algorítmico. De fato, nas medições o algoritmo permaneceu não significativo na maioria das topologias sob o critério pareado (variações de $-1{,}0\%$ a $+3{,}1\%$; Aleatório, Quase ordenado, Duplicados e Zigue-zague não significativos, Tartarugas $+3{,}1\%$ significativo na seed 42 mas instável entre *pools*), com exceção de uma degradação consistente no cenário *Invertido* em $n = 10.000$ (puro $19.150{,}38 \pm 353{,}49\,\mu s$ vs. com pré $20.900{,}45 \pm 113{,}30\,\mu s$), também reproduzida em $n = 100.000$ ($-12{,}8\%$, puro $1.849{,}65 \pm 8{,}34$ ms vs. com pré $2.086{,}10 \pm 8{,}82$ ms). Nas topologias adicionais (Serra, Tubo, Runs, Real), todas as variações ficam entre $-0{,}4\%$ e $+1{,}2\%$ e não significativas — uma confirmação limpa de neutralidade.

A degradação de ≈9–13% do *Selection Sort* no Invertido é real e reproduzível (também em $n{=}100.000$), embora modesta: como o número de comparações é fixo, esperava-se no máximo a eliminação das trocas. Contagens do *harness* instrumentado (material suplementar) mostram o mesmo total de comparações em todos os cenários, com atualizações de mínimo e trocas variando conforme a entrada, e a inspeção do assembly sugere custos assimétricos por caminho no laço interno. Sem contadores de hardware, se o mecanismo envolve predição de desvios ou apenas o *layout* do código gerado permanece hipótese. Em qualquer caso, sem ganho sistemático, o resultado confirma a premissa de que métodos não adaptativos não exploram a redução de inversões [Estivill-Castro and Wood, 1992]. Da mesma forma, em entradas aleatórias as condições de troca da varredura podem pressionar o preditor de desvios [Edelkamp and Weiß, 2016; Kaligosi and Sanders, 2006], mas isso permanece hipótese: onde a redução de inversões é modesta, o *overhead* $O(n)$ supera o benefício.

A Figura 2 sintetiza essa dinâmica nas dez topologias: aproveitamento expressivo nos métodos quadráticos adaptativos, resposta mista do *Merge Sort*, degradação sistemática do *Quicksort* de três vias e neutralidade do *Selection Sort* não adaptativo (com a exceção do cenário Invertido). A Tabela 9 complementa a análise para $n{=}10^6$ nos métodos quasilineares.

Tabela 9. Faixa de ganho do pré-processamento por método para vetores de $1.000.000$ de elementos (mínimo e máximo sobre as nove topologias medidas; Real excluído pelo teto do asset nativo). Os valores por célula, com marcas de significância, estão no material suplementar.

Método	Mínimo	Máximo
Merge	$-11{,}5\%$ (Tartarugas)	$+18{,}6\%$ (Zigue-zague)
Quick	$-37{,}7\%$ (Invertido)	$+5{,}5\%$ (Quase ordenado)
`sort_unstable`	$-194{,}3\%$ (Duplicados)	$+20{,}0\%$ (Zigue-zague)
`sort`	$-98{,}0\%$ (Duplicados)	$+57{,}5\%$ (Zigue-zague)

Os valores de $n{=}10^6$ replicam o padrão de $n{=}10.000$: ganhos do Merge Sort restritos a Zigue-zague ($+18{,}6\%$), Invertido ($+6{,}0\%$) e Duplicados ($+4{,}8\%$, significativo na seed 42 mas instável entre *pools*), degradação sistemática do Quick-

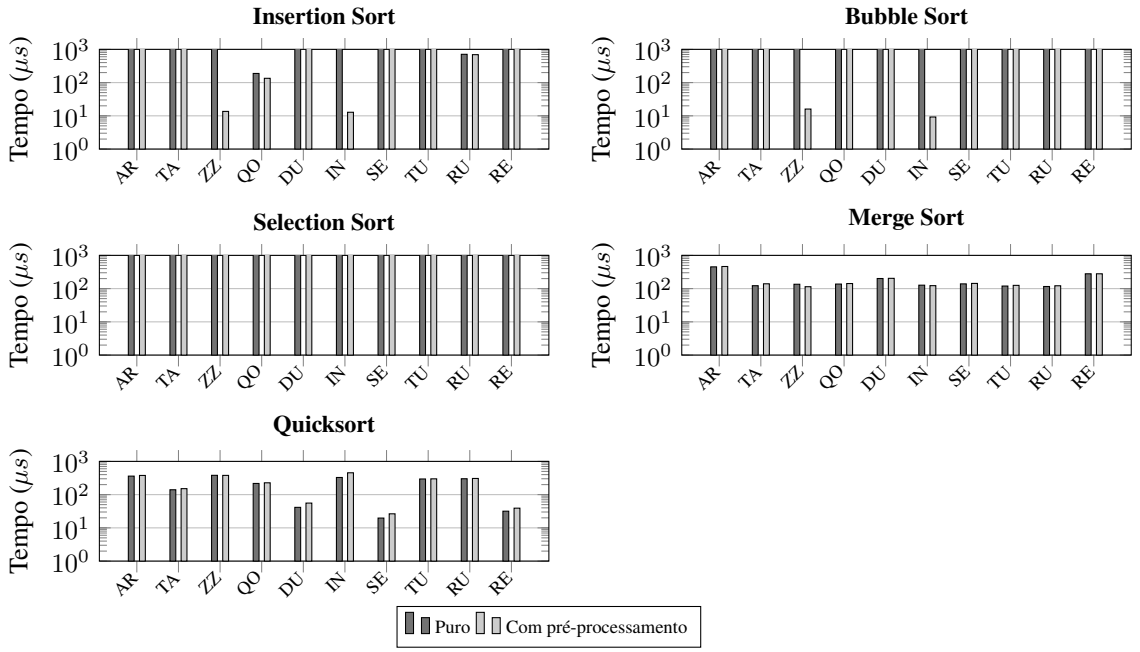


Figura 2. Comparação temporal entre a ordenação pura e com pré-processamento simétrico ($n = 10.000$) nas dez topologias, em escala logarítmica. As abreviações são: AR (Aleatório), TA (Tartarugas), ZZ (Zigue-zague), QO (Quase ordenado), DU (Duplicados), IN (Invertido), SE (Serra), TU (Tubo), RU (Runs) e RE (Real).

sort (até $-37{,}7\%$), com Tartarugas ($+0{,}0\%$) e Quase ordenado ($+5{,}5\%$) reprovados no teste pareado de robustez, confirmando a delimitação de aplicabilidade aos métodos adaptativos. Sob o critério pareado, as células não significativas em $n{=}10^6$ são Merge/Aleatório ($-0{,}9\%$) e Quicksort/Quase ordenado.

A Figura 3 estende o quadro por todos os tamanhos coletados ($n{=}16$–10^7; 10^7 apenas para quasilineares e baselines): *Insertion Sort*/Aleatório sobe de $\approx 0\%$ em $n{=}16$ a um platô estável de $\approx 44\%$ desde $n{=}10^4$; *Bubble Sort*/Aleatório é não monotônico em n pequeno (pico de $+31{,}9\%$ em $n{=}64$); *Merge Sort*/Zigue-zague cresce monotonicamente (1,8–18,6%), enquanto *Merge Sort*/Invertido é negativo em n pequeno, positivo no meio e reduz-se a $-2{,}2\%$ em 10^7; *Quicksort*/Aleatório atenua com o tamanho ($-10{,}6$ a $-1{,}8\%$); *Quicksort*/Quase ordenado é não monotônico e positivo apenas em 10^6 ($+5{,}5\%$, não robusto entre *pools*).

Contra a referência `sort_unstable`, o método é sistematicamente desfavorável em todos os tamanhos (Padrão/Aleatório de $-0{,}9\%$ em $n{=}16$ até $-77{,}5\%$ em $n{=}1.000$), e *Insertion Sort*/Serra só fica positivo acima de $n{=}128$ ($+17$–$+53\%$), com *Insertion Sort*/Tubo plano próximo de zero. Esses resultados evidenciam que as tendências em função do tamanho não se estabilizam uniformemente entre as topologias, de modo que os efeitos devem ser relatados separadamente por topologia.

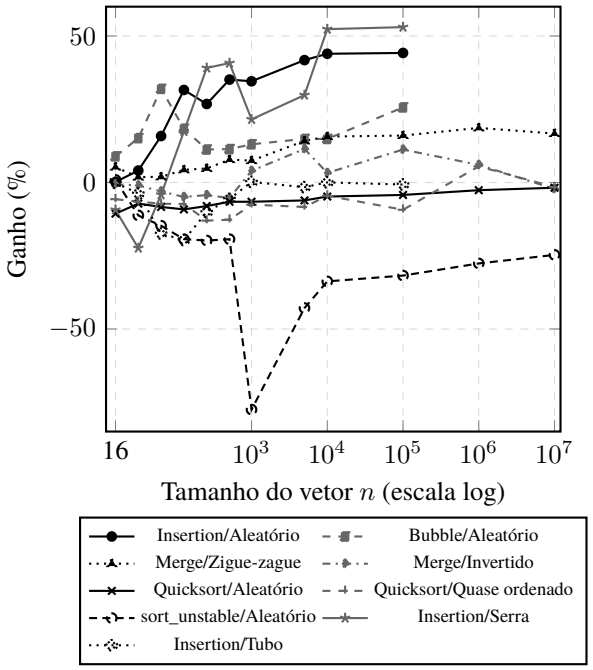


Figura 3. Ganho do pré-processamento em função do tamanho da entrada para pares algoritmo–topologia selecionados (médias do Criterion; métodos quadráticos limitados a $n{=}10^5$; pontos em $n{=}10^7$ apenas para quasilineares e baselines; lacunas são células não planejadas; valores por célula no material suplementar).

5 Considerações Finais

No contexto da ordenação de dados, cujo custo prático depende do grau de desordem da entrada, este trabalho apresentou e avaliou um pré-processamento simétrico $O(n)$ para redução de inversões. Em $n{=}10.000$, removeu 40–62% das inversões nas entradas não alinhadas (exceto Tubo e Runs, $\approx$0%), com ganhos de 14–60% no *Insertion Sort* e no *Bubble Sort*; a eliminação quase total e os *speedups* de $\times$386–2.280

ocorreram apenas nas entradas alinhadas por simetria (Invertido, Zigue-zague), desenhadas em torno da varredura, e não devem ser generalizados. Os ganhos se materializam quando $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$; para o *Insertion Sort*, isso assume a forma $\Delta I \gtrsim 5n$ (Seção 3.4). O método não supera o ordenador padrão em entradas aleatórias (`sort_unstable` $\approx$36$\times$ mais rápido), mas supera-o no Zigue-zague em n=10^4; a detecção de *runs* captura sozinha o extremo apenas no Invertido. O procedimento não preserva estabilidade, limitando o uso com registros cuja ordem de iguais importe.

O *Selection Sort* confirmou a hipótese de neutralidade (variações de $-1{,}0\%$ a $+3{,}1\%$, majoritariamente não significativas ou instáveis entre *pools*), com exceção de uma degradação real, porém modesta, de $\approx$9–13% na entrada invertida. Sem contadores de hardware, o mecanismo exato permanece hipótese (Seção 4.3). A invariância do número de comparações ($n(n-1)/2$) delimita que a redução de inversões não se converte em ganho para métodos não adaptativos.

Nos métodos $O(n \log n)$, a técnica não se mostrou benéfica na implementação avaliada. Com o *Quicksort* de três vias e mediana de três, houve degradação sistemática (4–8% na maioria das topologias; $\approx$31–38% em Duplicados e Invertido), em grande parte na escala do próprio *overhead*, além de degenerescência da base em entradas estruturadas mesmo sem pré-processamento (Seção 4.3). No *Merge Sort*, os ganhos restringiram-se a Zigue-zague e Invertido; o mecanismo provável (previsibilidade de desvios na intercalação) é hipótese. A viabilidade prática condiciona-se, portanto, ao *Insertion Sort* (e, com ressalvas, ao *Bubble Sort*) em entradas com alta desordem removível.

Como limitações, registram-se o viés de seleção das topologias autorais, as pseudo-réplicas determinísticas, o rótulo operacional "Real" e a ausência de ablação sistemática (passo espelhado isolado vs. ajustes) nas demais topologias e de baselines $O(n)$ alternativos.

Como trabalhos futuros, recomenda-se: (i) derivar a forma fechada do acoplamento espelho–adjacente (Seção 3.4) e validar a anomalia do *Quicksort* contra implementação de referência (pdqsort); (ii) repetir os testes em segunda máquina e em Linux com `perf_event` para medir *cache misses* e *branch mispredictions*; (iii) incluir ablação, topologias padrão da literatura, passada $O(n)$ alternativa e técnicas como *BlockQuicksort* [Edelkamp and Weiß, 2016] e *Timsort* [Auger *et al.*, 2018].

6 Uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho, os autores utilizaram ferramentas de inteligência artificial generativa como apoio à elaboração e à revisão do texto, à organização da análise dos resultados experimentais e à conferência das execuções. Essas ferramentas não foram utilizadas para gerar dados experimentais ou produzir resultados numéricos. Todo conteúdo, decisão metodológica, resultado e interpretação apresentados neste artigo foram conferidos, discutidos e validados pelos autores, que mantêm responsabilidade integral pela precisão, originalidade e integridade do trabalho. O uso de IA não substituiu o julgamento científico dos autores nem a responsabilidade destes sobre o conteúdo científico.

7 Repositório da Pesquisa

O código-fonte, os scripts de geração e de análise de dados, o *harness* de *benchmark*, o *asset* de bytes congelado utilizado na topologia Real e os arquivos CSV que fundamentam as tabelas e figuras deste artigo estão disponíveis publicamente no repositório `https://github.com/Zidan-09/presorting-algorithm`. O repositório reúne a implementação em Rust dos algoritmos de ordenação avaliados e do pré-processamento simétrico, além dos procedimentos experimentais necessários à reprodução dos resultados. A reprodução dos experimentos pode ser realizada por meio do comando `cargo bench`.

A Material suplementar

Os valores por célula (média $\pm$ meia-largura do IC 95% do Criterion, com marcas */† do delineamento pareado) que fundamentam as Tabelas 4, 6 e 9, bem como as contagens do *harness* instrumentado, os CSVs das figuras e a estabilidade entre *pools*, estão disponíveis no repositório da pesquisa (Seção 7), em tabelas suplementares versionadas.

Declarações complementares

Contribuições dos autores

S. P. Nascimento contribuiu para a concepção do estudo, implementação dos algoritmos, execução dos experimentos e redação do manuscrito. F. C. Rocha contribuiu para a orientação da pesquisa e revisão do manuscrito. Todos os autores leram e aprovaram o manuscrito final.

Conflitos de interesse

Os autores declaram que não têm nenhum conflito de interesses.

Disponibilidade de dados e materiais

Os conjuntos de dados (e/ou softwares) gerados e/ou analisados durante o estudo atual estão disponíveis em `https://github.com/Zidan-09/presorting-algorithm` (Seção 7).

Outras informações relevantes

Pesquisa experimental computacional sem sujeitos humanos; nenhum dado pessoal coletado. Experimentos reproduzíveis via `cargo bench` conforme a Seção 7. O uso de ferramentas de IA generativa é declarado na Seção 6.

Referências

Auger, N., Jugé, V., Nicaud, C., and Pivoteau, C. (2018). On the worst-case complexity of timsort. In *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, volume 112 of *LIPIcs*, pages 4:1–4:13. DOI: 10.4230/LIPIcs.ESA.2018.4.

Bentley, J. L. and McIlroy, M. D. (1993). Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265. DOI: 10.1002/spe.4380231105.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, 4th edition.

Edelkamp, S. and Weiß, A. (2016). Blockquicksort: Avoiding branch mispredictions in quicksort. In *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, volume 57 of *LIPIcs*, pages 38:1–38:16. DOI: 10.4230/LIPIcs.ESA.2016.38.

Estivill-Castro, V. and Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476. DOI: 10.1145/146370.146381.

Gil, A. C. (2019). *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1):10–16. DOI: 10.1093/comjnl/5.1.10.

Hwang, H.-K., Yang, B.-Y., and Yeh, Y.-N. (2000). Presorting algorithms: An average-case point of view. *Theoretical Computer Science*, 242(1-2):29–40. DOI: 10.1016/S0304-3975(98)00181-9.

Kaligosi, K. and Sanders, P. (2006). How branch mispredictions affect quicksort. *Journal of Experimental Algorithmics*, 11. Também apresentado em ESA 2006. DOI: 10.1145/1187436.1216588.

Knuth, D. E. (1973). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA.

Mannila, H. (1985). Measures of presortedness and optimal sorting algorithms. *IEEE Transactions on Computers*, C-34(4):318–325. Apresentado no ICALP 1984 (LNCS 172, pp. 324–336); versão estendida no periódico. DOI: 10.1109/TC.1985.5009382.

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993. DOI: 10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#.

Rust Standard Library (2026). Method `slice::sort_unstable` (ipnsort) and `slice::sort`. Disponível em: `https://doc.rust-lang.org/std/primitive.slice.html`. Acesso em 2026.

Sampieri, R. H., Collado, C. F., and Lucio, M. d. P. B. (2021). *Metodologia de pesquisa*. Penso.

Sedgewick, R. and Wayne, K. D. (2011). *Algorithms*. Addison-Wesley, Upper Saddle River, 4th edition.

Ziviani, N. (2021). *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning.